

```
import os
import logging
from datetime import datetime,
timedelta
from telegram import Update,
InlineKeyboardButton,
InlineKeyboardMarkup
from telegram.ext import
Application, CommandHandler,
ContextTypes
from dotenv import load_dotenv
import json

# Загружаем переменные окружения
load_dotenv()

# Настройка логирования
logging.basicConfig(
    format='%(asctime)s - %
(name)s - %(levelname)s - %
(message)s',
    level=logging.INFO
)
```

```
logger =
logging.getLogger(__name__)

# Константы
BOT_TOKEN =
os.getenv('BOT_TOKEN')
GROUP_ID =
int(os.getenv('GROUP_ID',
'-1003951214564'))
GENERATOR_URL =
os.getenv('GENERATOR_URL',
'https://fussreels.netlify.app/')
INITIAL_PASSWORD =
os.getenv('INITIAL_PASSWORD',
'Reelslegko')
ANTHROPIC_API_KEY =
os.getenv('ANTHROPIC_API_KEY')

# Хранилище паролей (в продакшене
использовать БД)
PASSWORD_FILE =
'password_data.json'

def load_password_data():
    """Загружает данные о пароле
из файла"""
```

```

        if
os.path.exists(PASSWORD_FILE):
        with open(PASSWORD_FILE,
'r') as f:
            return json.load(f)
        return {
            'password':
INITIAL_PASSWORD,
            'created_date':
datetime.now().isoformat(),
            'next_change':
(datetime.now() +
timedelta(days=30)).isoformat()
        }

def save_password_data(data):
    """Сохраняет данные о пароле
в файл"""
    with open(PASSWORD_FILE, 'w')
as f:
        json.dump(data, f,
indent=2)

def generate_new_password():
    """Генерирует новый пароль"""
    import random

```

```

import string

# Формат: ReelsXXXX (буквы +
цифры)
    random_part =
''.join(random.choices(string.ascii_letters + string.digits, k=8))
    return f"Reels{random_part}"

async def
check_password_expiry(context:
ContextTypes.DEFAULT_TYPE):
    """Ежедневная проверка
    необходимости смены пароля"""
    password_data =
load_password_data()
    next_change =
datetime.fromisoformat(password_data['next_change'])

    if datetime.now() >=
next_change:
        # Время менять пароль
        new_password =
generate_new_password()
        password_data['password']

```

```
= new_password

password_data['created_date'] =
datetime.now().isoformat()

password_data['next_change'] =
(datetime.now() +
timedelta(days=30)).isoformat()

save_password_data(password_data)

        logger.info(f"✅ Пароль
изменён на: {new_password}")

async def start(update: Update,
context:
ContextTypes.DEFAULT_TYPE) ->
None:
    """Команда /start"""
    user = update.effective_user
    logger.info(f"Пользователь
{user.id} (@{user.username})
начал диалог")

    welcome_text = (
        "👋 Привет! Я **Подоро Рилс
```

```
Генератор**\n\n"
    "Я помогу тебе
генерировать идеи для Reels о
подологии и ортопедии.\n\n"
    "Нажми /access чтобы
получить доступ к генератору!"
)

await
update.message.reply_text(welcome
_text, parse_mode='Markdown')

async def access(update: Update,
context:
ContextTypes.DEFAULT_TYPE) ->
None:
    """Команда /access - проверка
подписки и выдача доступа"""
    user = update.effective_user
    user_id = user.id

    logger.info(f"Пользователь
{user_id} запросил доступ")

    try:
        # Проверяем, подписан ли
```

```
пользователь на группу
    member = await
context.bot.get_chat_member(GROUP
_ID, user_id)

    # Статусы "подписчика":
'member', 'restricted',
'creator', 'administrator'
    if member.status in
['member', 'restricted',
'creator', 'administrator']:
        # Пользователь
подписан ✅
        password_data =
load_password_data()
        current_password =
password_data['password']

        success_text = (
            "✅ **Спасибо за
подписку!**\n\n"
            "🎬 **Вот твой
доступ к Генератору Рилс:**\n\n"
            f"🔗 Ссылка:
{GENERATOR_URL}\n"
            f"🔑 Пароль:
```

```
`{current_password}`\n\n"  
    "📝 Пароль  
меняется автоматически раз в  
месяц.\n\n"  
    "💡 **Как  
использовать:**\n\n"  
    "1. Открой ссылку  
выше\n"  
    "2. Введи  
пароль\n"  
    "3. Выбери  
аудиторию (новички/педикюры/  
подологи)\n"  
    "4. Нажми  
'Сгенерировать 5 идей'\n"  
    "5. Готовые идеи  
для Reels в руках! 🎉"  
    )  
  
    # Кнопка "Открыть  
генератор"  
    keyboard =  
    InlineKeyboardMarkup([  
  
    [InlineKeyboardButton("🎬 Открыть  
генератор", url=GENERATOR_URL)]
```



```
    ])  
  
    await  
update.message.reply_text(  
    success_text,  
  
    parse_mode='Markdown',  
  
    reply_markup=keyboard  
    )  
    logger.info(f"✅  
Доступ выдан пользователю  
{user_id}")  
  
    else:  
        # Пользователь не  
        подписан ❌  
        denied_text = (  
            "❌ **Ты ещё не  
            подписан на группу!**\n\n"  
            "Чтобы получить  
            доступ к Генератору Рилс, "  
            "сначала  
            подпишись на нашу группу  
            СТОГ.\n\n"  
            "После подписки
```

```
вернись и напиши /access"
    )

    keyboard =
    InlineKeyboardMarkup([

    [InlineKeyboardButton("📢 Перейти
    в группу СТ0Г",
    url="https://t.me/+N5Ihg4P01xozZG
    Qy")]

    ])

    await
    update.message.reply_text(
        denied_text,

    parse_mode='Markdown',

    reply_markup=keyboard
    )
    logger.warning(f"❌
    Пользователь {user_id} не
    подписан на группу")

    except Exception as e:
        error_text = (
```

```

        f"⚠️ Ошибка:
        {str(e)}\n\n"
        "Попробуй позже или
        напиши администратору."
    )
    await
    update.message.reply_text(error_t
    ext)

    logger.error(f"Ошибка при
    проверке подписки: {e}")

async def password(update:
Update, context:
ContextTypes.DEFAULT_TYPE) ->
None:
    """Команда /password -
    показать текущий пароль (для
    проверки)"""
    password_data =
    load_password_data()
    current_password =
    password_data['password']

    info_text = (
        f"🔑 **Текущий пароль:**
        `{current_password}`\n\n"

```

```

        f"📅 Создан:
        {password_data['created_date']
        [:10]}\n"

        f"🔄 Следующая смена:
        {password_data['next_change']
        [:10]}}"
    )

    await
    update.message.reply_text(info_text, parse_mode='Markdown')

    async def help_command(update:
    Update, context:
    ContextTypes.DEFAULT_TYPE) ->
    None:
        """Команда /help"""
        help_text = (
            "📖 **Доступные
            команды:**\n\n"
            "/start - Начать\n"
            "/access - Получить
            доступ к генератору (нужна
            подписка)\n"
            "/password - Показать
            текущий пароль\n"

```

```

        "/help - Эта справка"
    )
    await
update.message.reply_text(help_text, parse_mode='Markdown')

def main() -> None:
    """Запуск бота"""

    if not BOT_TOKEN:
        raise
ValueError("BOT_TOKEN не
установлен в переменных
окружения!")

    logger.info("🤖 Запуск бота
Подо Рилс Генератор...")

    # Создаём приложение
    application =
Application.builder().token(BOT_T
OKEN).build()

    # Добавляем обработчики
команд

```

```
application.add_handler(CommandHandler("start", start))
```

```
application.add_handler(CommandHandler("access", access))
```

```
application.add_handler(CommandHandler("password", password))
```

```
application.add_handler(CommandHandler("help", help_command))
```

```
    # Добавляем задачу проверки  
    пароля каждый день в 00:00
```

```
application.job_queue.run_daily(  
    check_password_expiry,  
    time=None, # Запускается  
    каждый день  
    days=tuple(range(7))  
)
```

```
    logger.info("✅ Бот готов к  
    работе!")
```

```
    logger.info(f"📱 Бот доступен  
    по адресу: t.me/podo_reels_bot")
```

```
# Запускаем бота
application.run_polling()

if __name__ == '__main__':
    main()
```